Benchmarking Spiking Reservoir Architectures for Efficient Univariate Time Series Classification

Favour Akpasi
Johannes Kepler University, Linz.

March 26, 2026

Abstract

Spiking Neural Networks (SNNs) offer a promising avenue for energy-efficient Information processing, especially when deployed on specialized neuromorphic hardware, e.g., SpiNNaker, Loihi, etc. SNNs employ event driven computation where neurons communicate only when a certain firing threshold is exceeded. Reservoir Computing (RC) mitigates training complexity by fixing internal weights, restricting training to a read-out layer. The combination of RC and SNN promises energy efficient information processing that mitigates both energy demand and training complexity. This work presents a benchmark of reservoir-based spiking architectures against a conventional Long Short-Term Memory (LSTM) network for the classification of electrocardiogram signals. Three distinct architectures were evaluated: a baseline LSTM, a stochastic Liquid State Machine (LSM) with fixed random connectivity, and a structured Legendre Spiking Neural Network (LSNN) utilizing deterministic weights derived from the Legendre Memory Unit (LMU) theory. Through a grid search of hyperparameters specifically the input scale and decay rate (β) for the LSM, and the Legendre dimension (d) for the LSNN, we analyze the trade-offs between classification accuracy, computational latency, spike sparsity and the impact of the reservoir's initialization strategy. Experimental results demonstrate that a structured LSNN ($d = 24$) achieves a peak test accuracy of 97.07%, closely matching the LSTM baseline of 97.24% and outperforming the random LSM at 89.96%. Analysis of spiking dynamics reveals that the LSNN leverages a dense-sparse coding strategy (71.26% sparsity) to encode temporal dependencies, while the LSM's higher sparsity (89.25%) correlates with insufficient information retention. While simulation on Von Neumann hardware resulted in higher latency for spiking models (30–40 ms/sample) compared to the LSTM (6.2 ms/sample), the high sparsity confirms the theoretical potential for significant energy savings on event-driven neuromorphic hardware. These findings indicate that full backpropagation through time is not strictly necessary for this task and incorporating mathematical structure into spiking reservoirs bridges the performance gap with conventional deep learning, enabling high-accuracy temporal processing compatible with the strict constraints of edge computing.

Contents

0.1 Introduction 2
0.2 Methodology 2
0.2.1 Dataset 2
0.2.2 Model Architectures 2
0.3 Experimental Setup 6
0.3.1 Hardware Environment 6
0.3.2 Hyperparameter Optimization 6
0.3.3 Sparsity and Event Metrics 7
0.3.4 Latency and Computational Cost 7
0.4 Results and Discussion 8
0.4.1 Classification Accuracy 8
0.4.2 Sparsity 8
0.4.3 Latency and the Simulation Bottleneck 9
0.4.4 Limitations 9
0.5 Conclusion 9

0.1 Introduction

Processing temporal data, such as electrocardiogram (ECG) signals, presents challenges in maintaining memory over time while minimizing computational costs. Although conventional Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) networks, have excelled for such tasks because of their ability to mitigate vanishing gradients, they are computationally intensive. This high resource demand hinders their deployment on edge devices such as wearable or implantable devices with strict size, weight, and power constraints [1–3].

This study addresses these challenges by comparing a conventional LSTM with two spiking reservoir models: A random spiking Liquid State Machine (LSM) and the structured Legendre Spiking Neural Network (LSNN). The work aims to answer two research questions: (1) Can reservoir-based classifiers achieve performance competitive with fully trained deep networks for ECG time-series classification? (2) How does the reservoir initialization strategy affect the performance of spiking reservoir architectures? These questions are used to evaluate whether spiking reservoirs, either randomly initialized or structurally defined, can approach the classification accuracy of an LSTM baseline while retaining the architectural simplicity and potential computational efficiency associated with neuromorphic approaches. To ensure a fair comparison, all architectures are evaluated on the same dataset under a common training and evaluation protocol. The comparisons are fair for relative benchmarking but not for equal parameter count or compute as seen in section 0.4.

0.2 Methodology

0.2.1 Dataset

The ECG5000 dataset from the UCR Time Series Archive was utilized for this experiment. It consists of 5000 electrocardiogram (heartbeat) sequences, each sampled at 140 Hz over a 1-second duration (140 time steps). Each sample represents the voltage signal recorded from a single ECG lead during one heartbeat cycle. The original dataset contains five distinct classes that represent different types of heartbeat. The initial exploratory analysis revealed an extreme class imbalance. Class 1 ("Normal") constitutes approximately 58.4% of samples, while classes 3, 4 and 5 are severely underrepresented (see Table 1). To ensure a proper evaluation of the capabilities of the different architectures rather than their handling of imbalances and following precedents in the literature [2], the problem was transformed into a binary classification task: Normal vs abnormal as seen below

- **Normal:** Original Class 1 retained as Class 0.
- **Abnormal:** Original Classes 2, 3, 4, and 5 aggregated into Class 1.

This results in a more balanced split suitable for benchmarking (see Table 2).

The implementation of all models and processing steps is publicly available on GitHub. [1]

0.2.2 Model Architectures

In this study, three sequence models for binary ECG classification were compared: a conventional LSTM, a random spiking reservoir in the form of a LSM, and a structured spiking model based on LDN features followed by a spiking hidden layer. All models receive an input sequence of length $T = 140$, with one scalar value per time step, such that each sample is represented as $\boldsymbol{x} \in \mathbb{R}^{T\times 1}$. The task is then to classify each input sequence as normal or abnormal.

[1] https://github.com/FavourAkpasi/Benchmarking-Spiking-Reservoir-Architectures

Table 1: Original dataset class distribution

Class	Training set	Test set
N	58.4% (292)	58.38% (2627)
R-on-T PVC	35.4% (177)	35.33% (1590)
PVC	2.00% (10)	1.91% (86)
SP	3.80% (19)	3.89% (175)
Unclassified beat (UB)	0.40% (2)	0.49% (22)

Note: The class distribution of the ECG5000 Dataset before binary processing. Abbreviations — N: Normal; R-on-T PVC: R-on-T premature ventricular contraction; PVC: Premature ventricular contraction; SP: Supra-ventricular premature beat; UB: Unclassified beat.

Table 2: Binary class distribution after class aggregation

Class	Training set	Test set
Normal (Class 0)	58.4% (292)	58.38% (2627)
Abnormal (Class 1)	41.6% (208)	41.62% (1873)

Note: The class distribution of the ECG5000 Dataset after binary processing. Class 0 corresponds to Normal beats (original Class 1), while Class 1 aggregates R-on-T PVC, PVC, SP, and UB (original Classes 2–5).

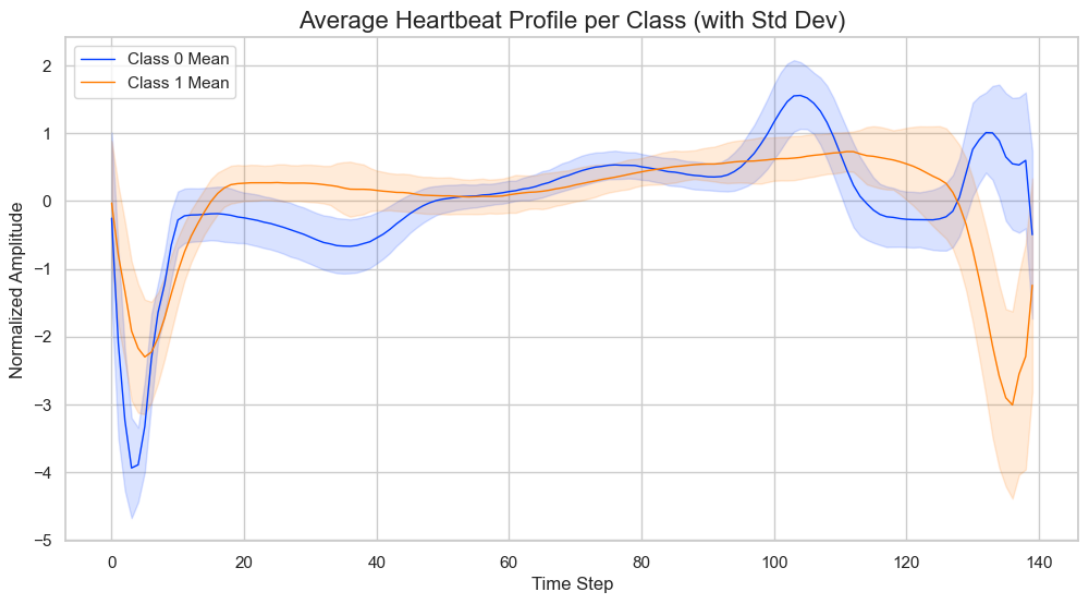

Figure 1: Average heartbeat profile per class (mean ± 1 standard deviation) from the processed dataset. The plot shows the class-wise mean normalized sample over the 140 time steps, with the shaded regions indicating one standard deviation across samples. The shape differences between the classes visually validate the feasibility of binary classification.

Model A: Long Short-Term Memory (LSTM)

A standard LSTM network was established to serve as the non spiking accuracy baseline. The LSTM uses gating mechanisms to control the information flow, allowing it to learn long-term dependencies [1]. Each input sample is represented as $\boldsymbol{x} \in \mathbb{R}^{T\times 1}$. The LSTM was implemented in pytorch as a two layer stacked architecture with hidden state dimensionality $h = 64$ using `batch_first` formatting. At each forward pass, the hidden and cell states are initialized to

zeros. The model processes the full ECG sequence and returns a hidden representation for each time step. The hidden representation from the final time step is retained as a fixed-length representation of the entire sequence. This representation is passed to a linear readout layer to produce class logits. The model parameters are trained using the adam optimizer with cross-entropy loss.

Model B: Liquid State Machine (LSM)

The LSM is a reservoir computing architecture that processes data using a recurrent network of spiking neurons followed by a trainable linear readout layer [4,5]. The reservoir is composed of N (in this case **64**) Leaky Integrate-and-Fire (LIF) neurons. The connections are defined by the input weight matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times N_{in}}$ and the recurrent weight matrix $\mathbf{W}_{\text{rec}} \in \mathbb{R}^{d \times d}$. The weights are initialized randomly and remain frozen during training to preserve the separation property and the fading memory of the system [4,6].

The dynamics of the reservoir are governed by the discrete-time LIF equations. At each time step t, the total synaptic current $\boldsymbol{u}_t$ is computed by integrating external inputs and recurrent spikes from the previous step:

$$\mathbf{u}_t = \mathbf{x}_t W_{\text{in}}^\top + \mathbf{s}_{t-1} W_{\text{rec}}^\top \tag{1}$$

where $\boldsymbol{x}_t \in \mathbb{R}^{B \times N_{in}}$ is the input batch and $\mathbf{s}_{t-1} \in \{0,1\}^{B \times d}$ is the spike vector from the previous step.

The membrane potential $\mathbf{m}_t$ changes according to a decay dynamics governed by the factor β:

$$\boldsymbol{m}_t = \beta \boldsymbol{m}_{t-1} + (1-\beta)\boldsymbol{u}_t - \boldsymbol{s}_{t-1} V_{th} \tag{2}$$

Here, $\beta \equiv e^{-\Delta t/\tau_m}$ represents the decay rate associated with the membrane time constant τ_m [7]. When the membrane potential exceeds the firing threshold V_{th}, a spike is emitted ($\mathbf{s}_t = 1$), and the potential is reset. To translate the temporal spike trains into a static representation for classification, we utilize a rate-coding readout scheme. The spike trains are aggregated over the time window T to form a mean firing rate vector $\mathbf{r} \in \mathbb{R}^{B \times d}$. Finally, the class logits $\mathbf{z} \in \mathbb{R}^{B \times C}$ are computed using a trainable linear readout:

$$\mathbf{z} = \boldsymbol{r}\mathbf{W}_{\text{out}}^\top + \boldsymbol{b}_{\text{out}} \tag{3}$$

where $\mathbf{W}_{\text{out}}$ and $\mathbf{b}_{\text{out}}$ are the only trainable parameters in the network and C is the number of classes. The LSM therefore separates temporal feature extraction, performed by the fixed spiking reservoir, from classification, performed by the learned readout.

Model C: Legendre Spiking Neural Network (LSNN)

The LSNN is a hybrid architecture that combines a non-spiking Legendre Delay Network (LDN) for temporal feature extraction with a spiking neural network for classification. The LDN layer approximates the input history using Legendre polynomials, efficiently compressing temporal information into a state vector of dimension d [2]. The LDN maintains a (d)-dimensional state vector ($\mathbf{h}_t$) that summarizes the recent history of the input signal. For each time step

$$\boldsymbol{h}_t = \boldsymbol{h}_{t-1}\bar{\mathbf{A}}^\top + \boldsymbol{x}_t\bar{\mathbf{B}}^\top \in \mathbb{R}^{B \times d} \tag{4}$$

where $\mathbf{h}_t$ is the zero initialized memory state. $\bar{\mathbf{A}} \in \mathbb{R}^{d \times d}$ and $\bar{\mathbf{B}} \in \mathbb{R}^{d \times 1}$ are the discretized versions of the continuous Legendre state-space matrices, calculated with a fixed time-step Δt. These weights are fixed (frozen) during training to ensure optimal orthogonal projection of the sliding window history. The continuous-valued features $\boldsymbol{h}_t$ are projected into a hidden layer of spiking neurons. The feature dimension is expanded to a hidden dimension of $3d$ [2]. A trainable encoder matrix $W_{\text{enc}} \in \mathbb{R}^{(3d) \times d}$ maps the Legendre features to synaptic currents:

$$\boldsymbol{u}_t = \boldsymbol{h}_t \mathbf{W}_{\text{enc}}^\top \in \mathbb{R}^{B \times 3d} \tag{5}$$

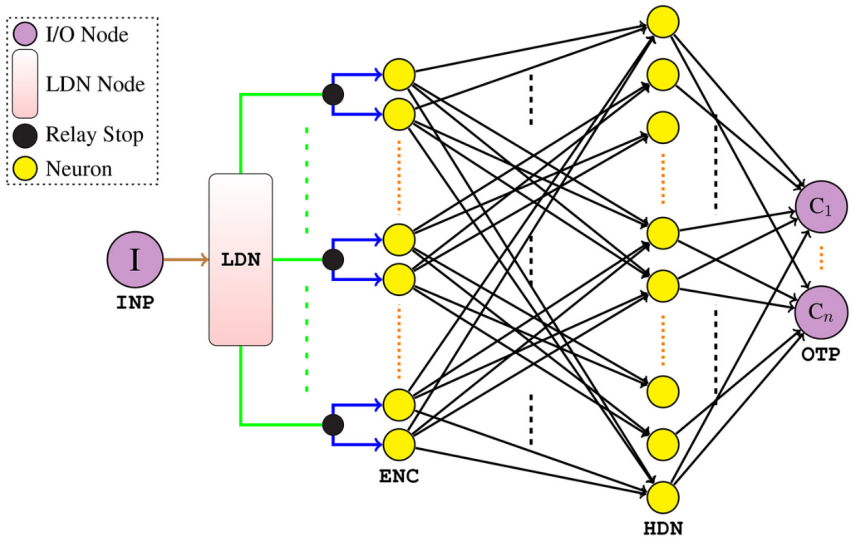

Figure 2: LSNN model architecture (all neurons are IF spiking). The INP is connected to the LDN through a static connection weight of 1. The outputs from the LDN, i.e., x(t) (the green connections) are relayed as is to the relay stops where each individual feature signal is multiplied by the static weights of 1 and 1 (simultaneously), i.e., the blue connections and fed to ENC. Only the connections in black are trained. adapted from [2]

The hidden layer consists of Leaky Integrate-and-Fire (LIF) neurons. The membrane potential $\mathbf{m}_t$ and the spike output $\boldsymbol{s}_t$ is then:

$$(\boldsymbol{s}_t, \boldsymbol{m}_t) = \mathrm{LIF}_\beta(\boldsymbol{u}_t, \boldsymbol{m}_{t-1}) \tag{6}$$

where β is the decay rate and $\boldsymbol{s}_t \in \{0,1\}^{B\times 3d}$ is the spike vector. To perform classification, we aggregate the spike trains into a rate-coded representation $\boldsymbol{r} \in \mathbb{R}^{B\times 3d}$ by averaging over the time dimension T. Finally, the class logits $\boldsymbol{z} \in \mathbb{R}^{B\times C}$ are obtained using a trainable linear readout layer:

$$\boldsymbol{z} = \boldsymbol{r}\mathbf{W}_{\text{out}}^\top + \boldsymbol{b}_{\text{out}} \tag{7}$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{C\times 3d}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^C$ are the trainable parameters. In contrast to the LSM, the temporal memory in this model is not created by random recurrence, but by a deterministic LDN state update. This allows the model to encode input history using a mathematically structured representation before spike-based classification.

Architectural differences

The three models differ mainly in how temporal information is represented. The LSTM learns its recurrent memory entirely through backpropagation. The LSM uses a fixed random spiking reservoir and trains only a readout layer. The structured LSNN replaces random reservoir dynamics with deterministic Legendre memory features and uses a trainable spiking hidden layer to convert those features into a discriminative spike-based representation. This comparison allows us to isolate the effect of reservoir structure and initialization strategy on ECG classification performance.

0.3 Experimental Setup

0.3.1 Hardware Environment

All experiments were conducted on an Apple MacBook Pro (M2, 16GB Unified Memory) which uses the ARM64 architecture and Apple's Metal Performance Shaders acceleration. This machine behaves differently from the NVIDIA GPUs (CUDA) used in most standard literature. The models were implemented in Python 3.9 using PyTorch 2.0. All models were trained using the Adam optimizer with Cross entropy loss for 30 epochs.

0.3.2 Hyperparameter Optimization

The selection of hyperparameters focused on the parameters most directly related to temporal memory, spike generation, and optimization stability. Because this was a practical benchmarking study rather than an exhaustive architecture search, the search space was kept small and mainly guided by prior literature, especially the Legendre-based spiking model proposed in [2], as well as by the available resources for computation. A common training budget of 30 epochs was used for all models. The Adam optimizer and cross-entropy loss were used across all models. For the spiking models, we searched over a small set of hyperparameters that directly control reservoir dynamics or structured temporal encoding. The comparative performance of the top configurations is detailed in Table 3.

Table 3: Grid Search Summary

Model	Hyperparameters	LR	Acc. (%)
LSNN (Structured)	$d = 24$, Gain=1.0	0.005	**97.07**
LSNN (Structured)	$d = 12$, Gain=1.0	0.005	93.84
LSNN (Structured)	$d = 6$, Gain=1.0	0.005	89.76
LSM (Random)	Scale=10.0, $\beta = 0.9$	0.005	89.96
LSM (Random)	Scale=50.0, $\beta = 0.9$	0.005	87.49

Note: LR denotes Learning Rate. For the Structured LSNN, d represents the dimensionality of the Legendre memory basis. For the Random LSM, Scale refers to the input weight scaling factor, and β indicates the membrane potential decay rate of the LIF neurons.

LSM hyperparameters

The LSM relies on the so called "edge of chaos" transition to separate inputs [8, 9], the search varied the input scaling factor (scale), the membrane decay constant (β), and the learning rate (LR). The scale controls the magnitude of the external current injected into the reservoir. Too small scale may leave the reservoir insufficiently active, whereas an excessively large drive may saturate the spiking activity and reduce class-separating dynamics. To show the effect of moderate and strong excitation, the scale was evaluated in 2 levels, 10.0 and 50.0. β determines how quickly the membrane potential decays over time and therefore controls the reservoir's fading-memory behavior. Since the ECG signals contain short but temporally meaningful waveform transitions over 140 time steps, we tested $\beta \in 0.9, 0.95$ to compare a more responsive reservoir against one with longer memory retention. The learning rate was varied over (0.001, 0.005) to balance convergence stability and training speed under the fixed epoch budget [2, 3, 10].

LSNN hyperparameters

The LSNN key hyperparameters are the Legendre dimension (d), the encoder initialization gain, and the learning rate. The Legendre dimension determines the dimensionality of the fixed LDN state and therefore the capacity of the model to represent recent input history. We

tested $d \in 6, 12, 24$ to examine how increasing structured memory capacity affects classification performance while keeping the model compact. The encoder initialization gain (varied between 1.0, 4.0) controls the scale of the trainable projection from the LDN features into the spiking hidden layer. This parameter was included because spiking models are sensitive to the magnitude of their input currents: a very small initialization may under-activate neurons, while an overly large initialization may lead to dense or unstable firing. Similar to the LSM, the learning rate was varied over (0.001, 0.005).

Grid Search design choices

Some parameters were kept constant to keep the comparison focused. In the LSM, the reservoir size was fixed at 64 neurons. In the structured LSNN, the hidden spiking layer size was fixed at $(3d)$, following the scaling strategy used in the reference architecture [2]. The LDN memory window was set to $\theta = 140$ so that the structured memory spans the entire length of the ECG sequence. That way, the Legendre state can represent information from the complete heartbeat rather than only a shorter temporal segment. The selected search grid was designed to test clear hypotheses about the behavior of the two spiking approaches, not intended to exhaust all possible configurations. For the LSM, the goal was to study whether a random reservoir benefits more from moderate or strong input drive and from shorter or longer fading memory. For the structured LSNN, the goal was to determine whether increasing the dimensionality of the Legendre memory representation improves classification and whether spike generation is sensitive to the initial encoder scale. The resulting trends support these design choices. The best LSM performance was obtained with the smaller input scale and lower decay setting, suggesting that moderate excitation and more responsive dynamics were better suited to the classification task. For the structured LSNN, the best performance was achieved at (d = 24) with gain (= 1.0), indicating that richer structured temporal memory was beneficial, while overly large encoder initialization was not as can be observed in Figure 3. Across both spiking models, the better configurations also favored the larger learning rate of (0.005), which likely improved convergence within the fixed number of epochs.

0.3.3 Sparsity and Event Metrics

An advantage of SNNs is their ability to process information sparsely, communicating only when significant events occur. To quantify this, we accumulated the output spike tensors for every sample in the test set. We defined Spike Density as the fraction of the total possible spike events ($neurons \times timesteps$) that actually caused a spike. Sparsity was calculated as $(1 - SpikeDensity) \times 100$. Additionally, we tracked Spikes Per Sample to estimate the total synaptic workload required for a single inference. These metrics serve as a proxy for dynamic power consumption on neuromorphic hardware, where energy is only consumed during spike events [7, 11].

0.3.4 Latency and Computational Cost

To observe and benchmark computational overhead, we measured both training time and inference latency. Training time was recorded for the entire 30-epoch cycle. For inference latency, we measured the wall-clock time required to process a single sample. To ensure robust measurements, we utilized a "warmup" phase to stabilize the GPU state, followed by repeated forward passes to calculate the average millisecond (ms) latency per sample. This approach highlights the distinction between algorithmic time steps and hardware execution time [10, 12]

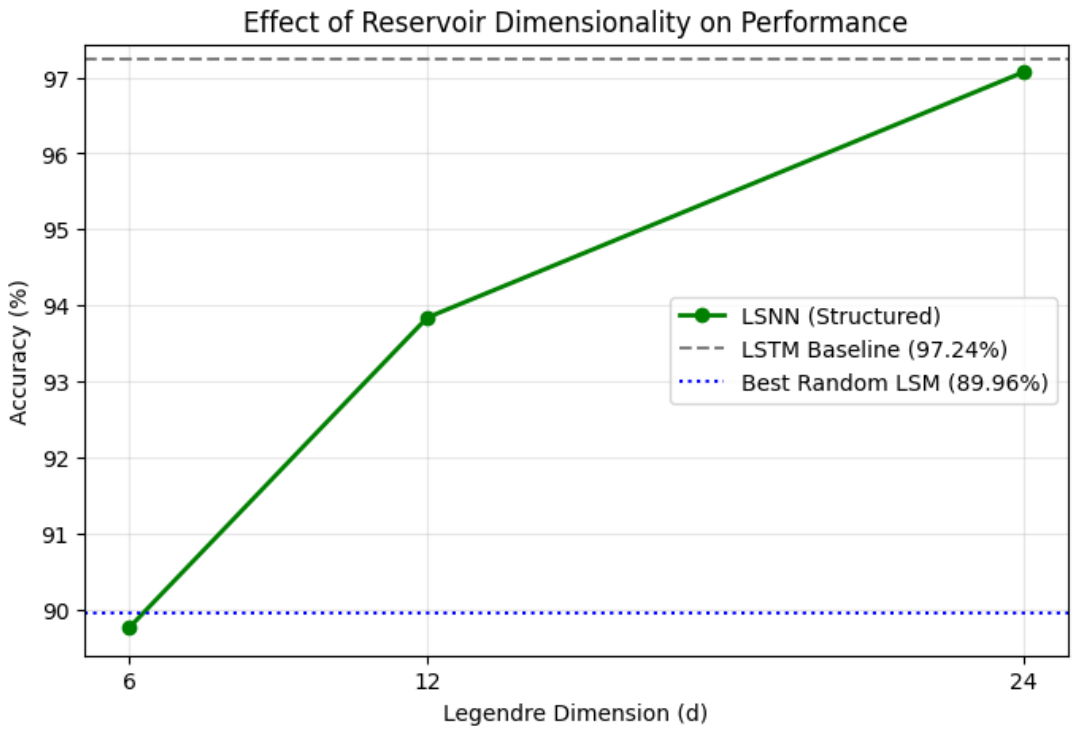

Figure 3: The effect of Reservoir dimensionality d on performance (accuracy). The accuracy of the structured LSNN improved as the d increased from 6 to 24

Table 4: Comprehensive Performance Benchmark: Non-Spiking Baseline vs. Spiking Reservoirs

Model Architecture	**Acc. (%)**	**Train Time (s)**	**Inf. Latency (ms)**	**Spikes / Sample**	**Sparsity (%)**
Model A: LSTM	97.24	18.74	6.20	N/A	N/A
Model B: LSM	89.96	78.82	30.64	963	89.3
Model C: LSNN	97.07	100.54	40.01	2,897	71.3

Note: The table shows comprehensive benchmark comparing spiking models across performance, and efficiency metrics. Training time is measured in seconds and Inference latency is measured in miliseconds.

0.4 Results and Discussion

0.4.1 Classification Accuracy

From the results on Table 4, we observe that the Structured LSNN achieved a maximum accuracy of 97.07% (with d=24), closely matching the Baseline LSTM (97.24%). In contrast, the Random LSM achieved 89.96%. This disparity highlights the effectiveness of the LDN structure in the LSNN; by mathematically deriving weights to orthogonalize the input history, the LSNN captures long-term dependencies that the random projections of the LSM fail to retain. The LSM's reliance on random connectivity necessitates a much larger reservoir to achieve the separation property required for high accuracy, whereas the LSNN achieves superior results with a compact and structured formulation [2].

0.4.2 Sparsity

The spike metrics as seen in Table 4 reveal a significant trade-off between sparsity and accuracy.

LSM

Exhibited high sparsity (89.25%) and low spike density (0.107). Although theoretically energy-efficient, this extreme sparsity implies that the reservoir often failed to propagate sufficient information through time, leading to lower accuracy.

LSNN

Showed moderate sparsity (71.26%) and higher spike density (0.287). The LSNN generated approximately 3x more spikes per sample (2897 vs. 962) than the LSM. This increased activity in the LSNN suggests that the structured weights successfully drove the neurons to encode the continuous-time history of the ECG signal into dense spike trains [10]. While this results in higher theoretical energy consumption than the LSM, it is still significantly more efficient than the continuous-valued activations of the LSTM, which effectively has 0% sparsity (processing dense floating-point numbers at every step).

0.4.3 Latency and the Simulation Bottleneck

The latency results as seen in Table 4 illustrate the so called "timescale mismatch" challenge when simulating SNNs on Von Neumann architectures [13]. The SNN models were approximately 5 to 6 times slower than the LSTM during inference. This is consistent with other literature noting that SNNs are computationally expensive to simulate on standard GPUs because the differential equations for every neuron must be solved iteratively for every time step (T=140), while LSTMs are highly optimized for parallel matrix operations [4]. The LSNN's higher latency compared to the LSM is attributed to the additional matrix multiplications required for the Legendre projection at each step [2]. However, it is important to note that on dedicated neuromorphic hardware (e.g., Loihi, SpiNNaker), this relationship often inverts, as the hardware processes spikes asynchronously and in parallel, decoupling execution time from the number of simulation steps [12, 14].

0.4.4 Limitations

This study has some limitations. First, the hyperparameter selection was based on repeated evaluation on the test set during training, which means the test set effectively served as a validation set. As a result, the reported test accuracies should be interpreted as optimistic estimates rather than strictly unbiased generalization performance. Second, the experiments were conducted on a single dataset and under a limited hyperparameter search space, so the conclusions are strongest for ECG5000 and should not be overgeneralized to all time-series classification tasks. Third, the structured LSNN evaluated here is a simplified implementation inspired by [2], rather than a reproduction of the original architecture, and therefore the results should be interpreted as evidence for the usefulness of structured temporal memory rather than as a direct replication study. Finally, efficiency conclusions are limited to training time, inference latency, and spike-based sparsity measured on conventional hardware; direct energy measurements on neuromorphic hardware were beyond the scope of this project.

0.5 Conclusion

This study compared the effectiveness of reservoir-based Spiking Neural Networks with standard Deep Learning for time-series classification. Using the ECG5000 dataset under a shared experimental protocol, we confirmed that reservoir-based classifiers can achieve performance competitive with fully trained deep networks, and that reservoir initialization strategy affects the performance of spiking reservoir architectures.

A structured reservoir-based spiking model can approach the performance of a fully trained deep recurrent baseline. The structured LSNN achieved 97.07% accuracy, closely matching the LSTM baseline at 97.24%, while substantially outperforming the random LSM. This indicates that full recurrent backpropagation is not strictly necessary to obtain strong classification performance on this task when the temporal structure is encoded effectively.

The initialization strategy of the reservoir has a clear effect on performance. The random LSM, which relies on frozen stochastic projections, achieved lower accuracy and higher sparsity, suggesting that its reservoir activity was often too weak or insufficiently informative for the classification task. In contrast, the structured LSNN benefited from deterministic Legendre-based temporal encoding, which provided a richer and more reliable representation of input history. These findings support the view that reservoir structure matters, and that analytically defined temporal memory can substantially reduce the performance gap between spiking models and conventional deep networks.

Beyond accuracy, the results also highlight an important trade-off. On conventional hardware, the spiking models incurred higher training and inference time than the LSTM because their neuron dynamics must be simulated sequentially across time. Nevertheless, the observed spike sparsity indicates that such models retain the potential for efficient event-driven computation when deployed on neuromorphic hardware. In this sense, the main contribution of the study is not simply that a spiking model performed well, but that a carefully structured spiking architecture was able to combine competitive accuracy with properties that remain attractive for low-power temporal processing.

The findings suggest that structured spiking reservoirs are a promising middle ground between conventional deep learning and fully random reservoir computing. They preserve the simplicity of fixed temporal feature extraction while achieving performance close to that of a fully trained LSTM. Future work should strengthen the evaluation protocol by introducing a dedicated validation split, repeating experiments across multiple random seeds, reporting additional classification metrics, and extending the analysis to direct neuromorphic deployment and energy measurement.

Bibliography

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] R. Gaurav, T. C. Stewart, and Y. Yi, "Reservoir based spiking models for univariate time series classification," *Frontiers in Computational Neuroscience*, vol. 17, 2023. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fncom.2023.1148284

[3] A. Fountain and C. Merkel, "Effect of biologically-motivated energy constraints on liquid state machine dynamics and classification performance," *Neuromorphic Computing and Engineering*, vol. 2, no. 2, p. 024005, apr 2022. [Online]. Available: https://doi.org/10.1088/2634-4386/ac5d1f

[4] V. Kholkin, O. Druzhina, V. Vatnik, M. Kulagin, T. I. Karimov, and D. N. Butusov, "Comparing reservoir artificial and spiking neural networks in machine fault detection tasks," *Big Data Cogn. Comput.*, vol. 7, p. 110, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:259241463

[5] W. Maass, T. Natschläger, and H. Markram, "Real-time computing without stable states: A new framework for neural computation based on perturbations," *Neural Computation*, vol. 14, no. 11, pp. 2531–2560, 2002.

[6] H. Jaeger, "The "echo state" approach to analysing and training recurrent neural networks-with an erratum note," *GMD Report*, vol. 148, 2001.

[7] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[8] G. Tanaka, T. Yamane, J. B. Héroux, R. Nakane, N. Kanazawa, S. Takeda, H. Numata, D. Nakano, and A. Hirose, "Recent advances in physical reservoir computing: A review," *Neural Networks*, vol. 115, pp. 100–123, 2019.

[9] S. Karki, D. C. Arana, A. Sornborger, and F. Caravelli, "Neuromorphic on-chip reservoir computing with spiking neural network architectures," 2024. [Online]. Available: https://arxiv.org/abs/2407.20547

[10] A. Voelker, I. Kajić, and C. Eliasmith, "Legendre memory units: Continuous-time representation in recurrent neural networks," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, Eds., 2019, pp. 15 570–15 579.

[11] S. Dey, D. Banerjee, A. M. George, A. Mukherjee, and A. Pal, "Efficient time series classification using spiking reservoir," in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.

[12] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, M. Lolis, A. Lu, A. McCoy, R. Paul, J. Tse, G. Vigh, B. Weng, A. Wild, Y. Yang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[13] T. Liu, "Harnessing slow dynamics in neuromorphic computation," 2019. [Online]. Available: https://arxiv.org/abs/1905.12116

[14] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, "The spinnaker project," *IEEE Transactions on Computers*, vol. 62, no. 11, pp. 2454–2467, 2013.